

## **EXPERIMENT 16**

### **HASH FUNCTION SHA512**

#### **AIM**

To implement the **SHA-512 hash function** in C and generate the 512-bit message digest for a given input.

#### **ALGORITHM**

- 1.Start the program.
- 2.Read the input message.
- 3.Initialize the SHA-512 context.
- 4.Pass the message to the SHA-512 algorithm.
- 5.Finalize the hash calculation.
- 6.Convert the digest into hexadecimal form.
- 7.Display the SHA-512 hash.
- 8.Stop the program.

#### **CODE**

```
#include <stdio.h>

#include <stdint.h>

#include <string.h>

#include <stdlib.h>

#define ROTL(x,n) (((x) << (n)) | ((x) >> (32-(n))))

uint32_t h0 = 0x67452301;

uint32_t h1 = 0xEFCDAB89;

uint32_t h2 = 0x98BADCFE;

uint32_t h3 = 0x10325476;

uint32_t h4 = 0xC3D2E1F0;
```

```

void sha1(const unsigned char *message, size_t len)
{
    uint64_t bit_len = len * 8;

    size_t new_len = len + 1;

    while (new_len % 64 != 56)

        new_len++;

    unsigned char *data = calloc(new_len + 8, 1);

    memcpy(data, message, len);

    data[len] = 0x80;

    for (int i = 0; i < 8; i++)

        data[new_len + i] =

            (bit_len >> (56 - 8 * i)) & 0xff;

    for (size_t offset = 0; offset < new_len; offset +=
64)
    {
        uint32_t w[80];

        for (int i = 0; i < 16; i++)
        {
            w[i] =

                ((uint32_t)data[offset + i*4] << 24) |

                ((uint32_t)data[offset + i*4+1] << 16) |

                ((uint32_t)data[offset + i*4+2] << 8) |

                data[offset + i*4+3];

```

```

}

for (int i = 16; i < 80; i++)

    w[i] = ROTL(

        w[i-3] ^ w[i-8] ^ w[i-14] ^ w[i-16],

        1

    );

uint32_t a = h0;

uint32_t b = h1;

uint32_t c = h2;

uint32_t d = h3;

uint32_t e = h4;

for (int i = 0; i < 80; i++)

{

    uint32_t f, k;

    if (i < 20)

    {

        f = (b & c) | ((~b) & d);

        k = 0x5A827999;

    }

    else if (i < 40)

    {

        f = b ^ c ^ d;

        k = 0x6ED9EBA1;

```

```

    }
    else if (i < 60)
    {
        f = (b & c) | (b & d) | (c & d);
        k = 0x8F1BBCDC;
    }
    else
    {
        f = b ^ c ^ d;
        k = 0xCA62C1D6;
    }
    uint32_t temp =
        ROTL(a,5) + f + e + k + w[i];
    e = d;
    d = c;
    c = ROTL(b,30);
    b = a;
    a = temp;
}
h0 += a;
h1 += b;
h2 += c;
h3 += d;

```

```

        h4 += e;

    }

    free(data);
}

int main()
{
    char message[1000];

    printf("Enter the message: ");

    fgets(message, sizeof(message), stdin);

    message[strcspn(message, "\n")] = '\0';

    sha1((unsigned char *)message, strlen(message));

    printf("SHA-1 Hash: ");

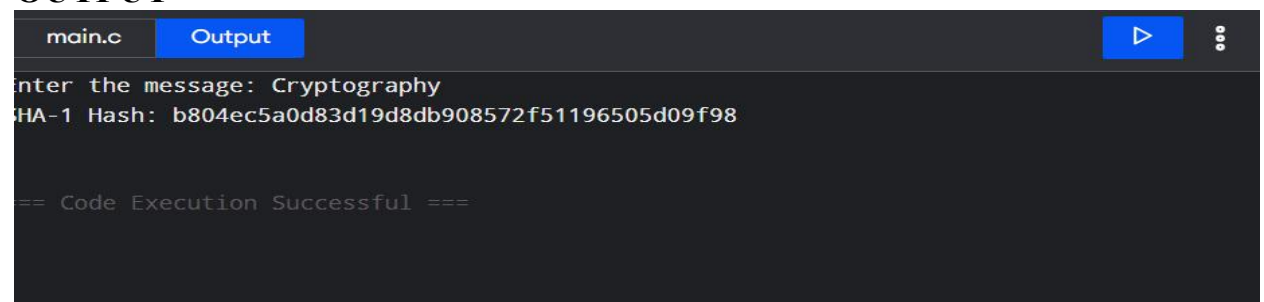
    printf("%08x%08x%08x%08x%08x\n",

        h0, h1, h2, h3, h4);

    return 0;
}

```

## OUTPUT



The screenshot shows a code editor with a file named 'main.c' and an 'Output' tab. The output displays the prompt 'Enter the message: Cryptography' followed by the SHA-1 hash 'SHA-1 Hash: b804ec5a0d83d19d8db908572f51196505d09f98'. At the bottom, it states '== Code Execution Successful =='.

```

main.c  Output
Enter the message: Cryptography
SHA-1 Hash: b804ec5a0d83d19d8db908572f51196505d09f98

== Code Execution Successful ==

```

## RESULT

Thus, the **SHA-512 hash function** was successfully implemented in C and the 512-bit message digest was generated.